FEATURE OVERVIEW

Frapper is completed project template with all features which are commonly used in projects.

Technologies ASP.NET Core 3.1, Entity Framework Core 3.1, jQuery, Bootstrap, SQL Server

Feature Overview

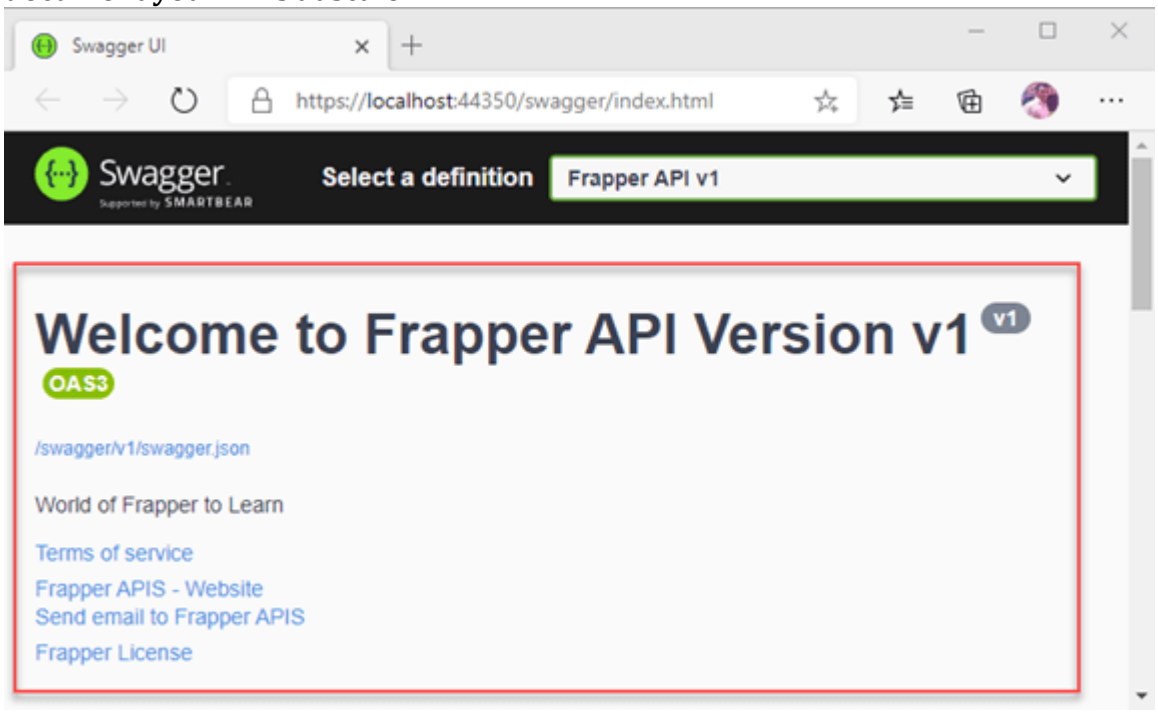
- Swagger
- Versioning
- Using JWT Token
- CRUD Operation
- Paging
- Validation
- Custom Middleware
- Request and Response Logging
- Exception Handling and Error Logging
- Common Response
- Using Unit of Work with Entity Framework Core and Dapper
- How to configure and run
- Clone code from Github: git clone Link
- Open solution Frapper.API.sln in Visual Studio 2019
- Database Scripts FrapperAPIDB (Main Database), Download Database Script
- Run Database Script which is provided
- appsettings.json file update DatabaseConnection (FrapperAPIDB Database)
- Make Changes in ConnectionStrings in appsettings.json file
- Build project which will restore all NuGet Packages
- Final Step Run Project

Swagger

For Designing API documentation, we have Used Swagger, If you see below Swagger documentation, you will find many details about these APIs such as Name, Url, Email, Version, Title, Description, terms of service, Contact, License.

What is Swagger?

Simplify API development for users, teams, and enterprises with the Swagger open source and professional toolset. Find out how Swagger can help you design and document your APIs at scale.



Version

To do versioning in ASP.NET Core WEBAPI, We have installed package from NuGet**Microsoft.AspNetCore.Mvc.Versioning**

After installing packages, we are going to register **AddApiVersioning** Service in ConfigureServices class.

```
services.AddApiVersioning(x :ApiVersioningOptions =>
{
    x.DefaultApiVersion = new ApiVersion( majorVersion: 1,  minorVersion: 0);
    x.AssumeDefaultVersionWhenUnspecified = true;
    x.ReportApiVersions = true;
    x.ApiVersionReader = new HeaderApiVersionReader( params headerNames: "x-frapper-api-version");
});
```

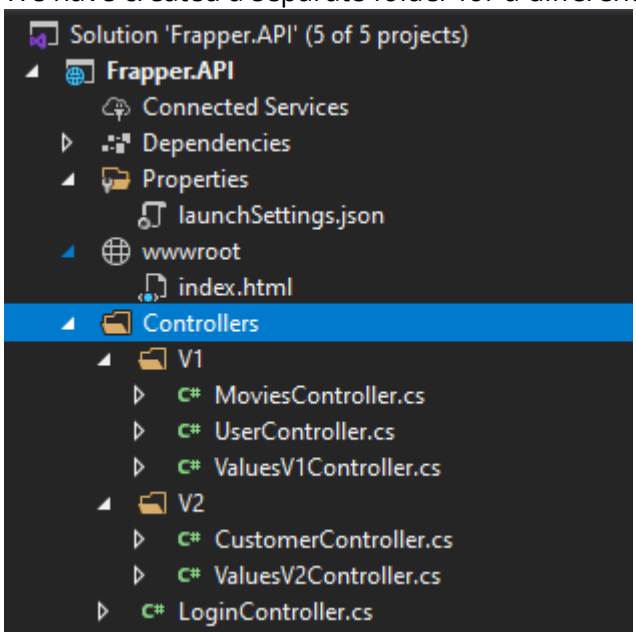
After registering Service, we can use **ApiVersion** Attribute and **MapToApiVersion** Attribute and specify a version.

```
namespace Frapper.API.Controllers.V1
{

    [Authorize (Roles = "User")]
    [Route( template: "api/values")]
    [ApiVersion("1.0",Deprecated = true)]
    [ApiController]
    0 references
    public class ValuesV1Controller : ControllerBase
    {
        // GET: api/<ValuesController>
        [HttpGet]
        [MapToApiVersion("1.0")]
        0 references
        public IEnumerable<string> Get()
        {
            return new string[] { "value1", "value2" };
        }
```

```
namespace Frapper.API.Controllers.V2
{
    [Authorize(Roles = "User")]
    [Route( template: "api/values")]
    [ApiVersion("2.0")]
    [ApiController]
    0 references
    public class ValuesV2Controller : ControllerBase
    {
        // GET: api/<ValuesController>
        [HttpGet]
        [MapToApiVersion("2.0")]
        0 references
        public IEnumerable<string> Get()
        {
            return new string[] { "value3", "value4" };
        }
```

Folder Structure

We have created a separate folder for a different version of APIs.



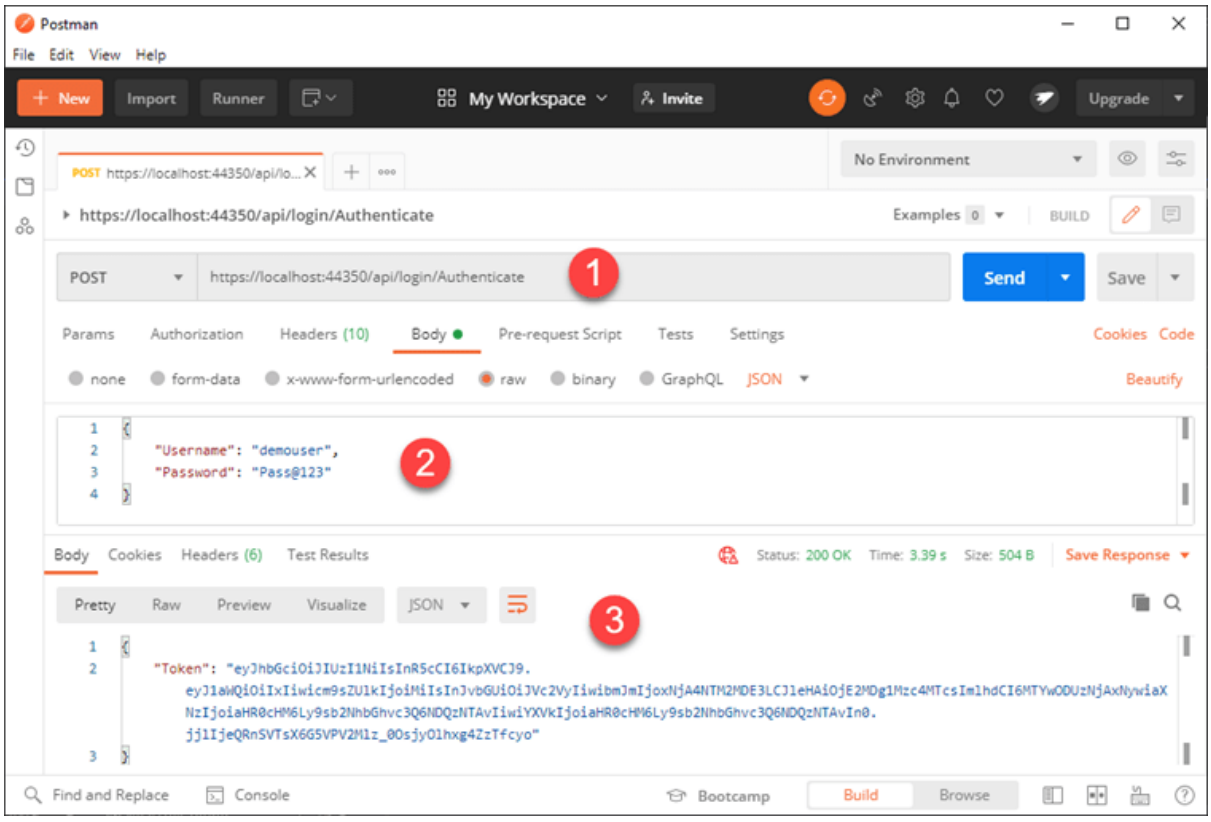
Using JSON Web Token (JWT)

Authentication

In this part, we will see Authentication in details we are going pass Username and Password if credentials are valid then in return, we will receive json web token (JWT token), We are going to create User and share their credentials to access our APIs.

Results | Messages

	UserId	UserName	FirstName	LastName	EmailId	MobileNo	Gender	Status	CreatedBy
1	1	demouser	Saineshwar	Bageri	demouser@frapper.com	9999999999	M	1	1
2	2	demoadmin	demoadmin	demoadmin	demoadmin@frapper.com	9998889998	M	1	1

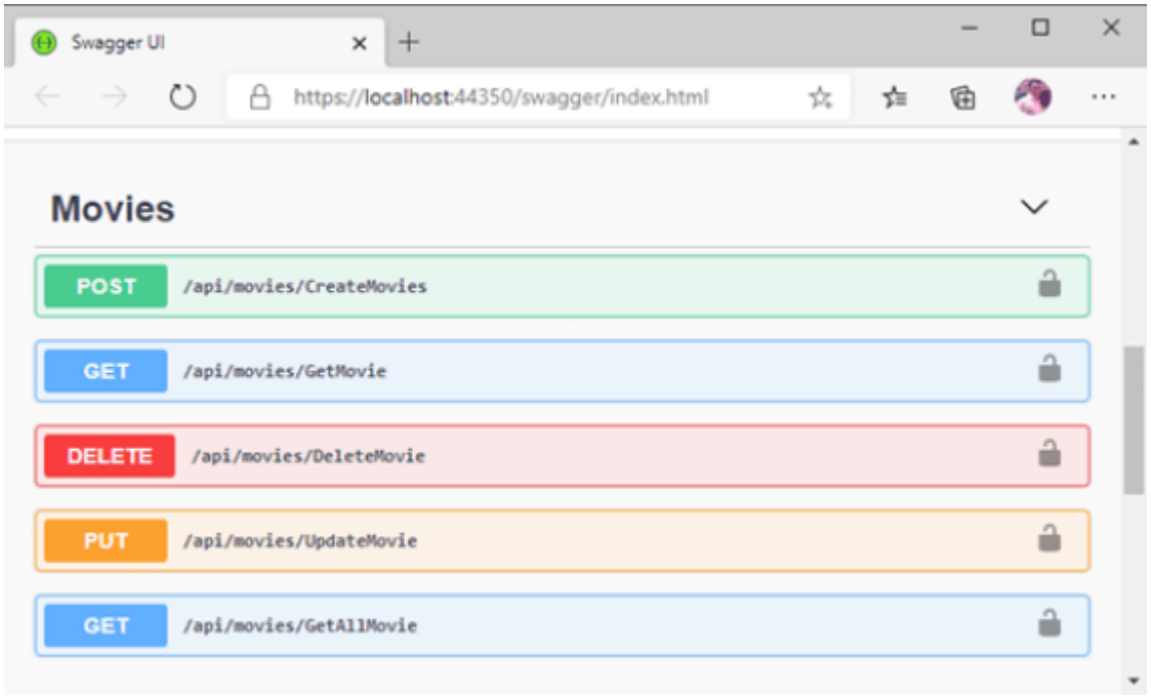


After getting token now let's use it for performing crud operations.

CRUD Operation

In this part, we are going to perform CRUD operation.

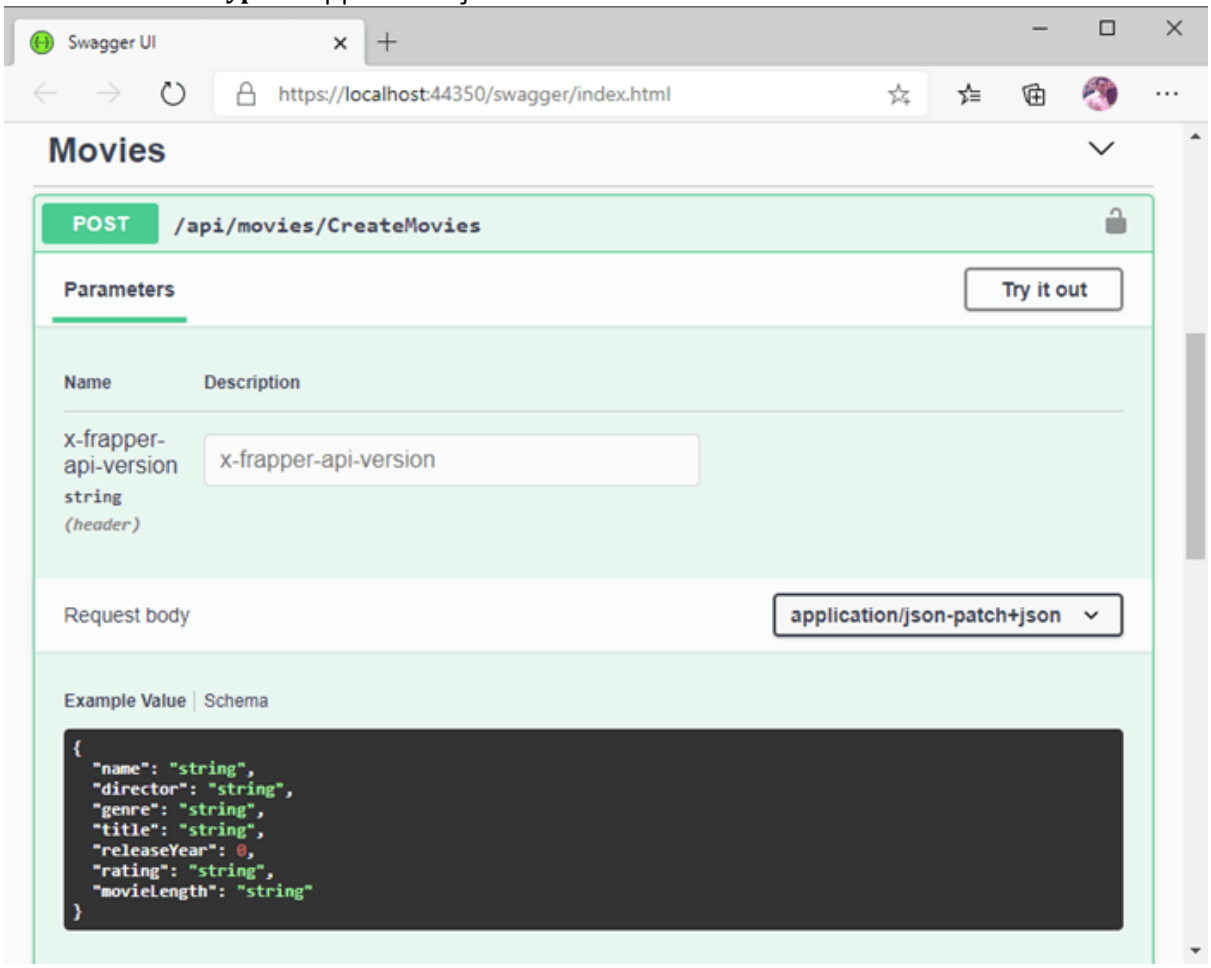
- Creating Movie
- Getting Movie Details by Id
- Deleting Movie Details
- Updating Movie Details
- Get All Movies



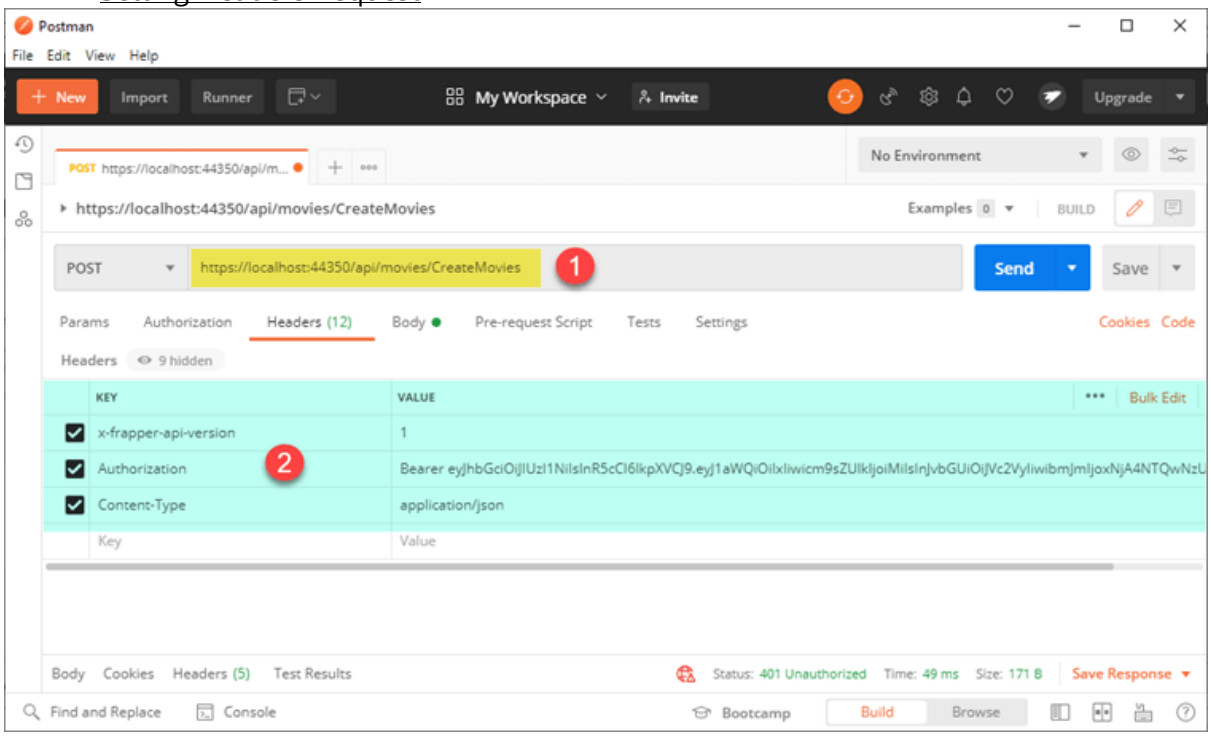
Creating Movie

For creating Movie, we need to send below headers

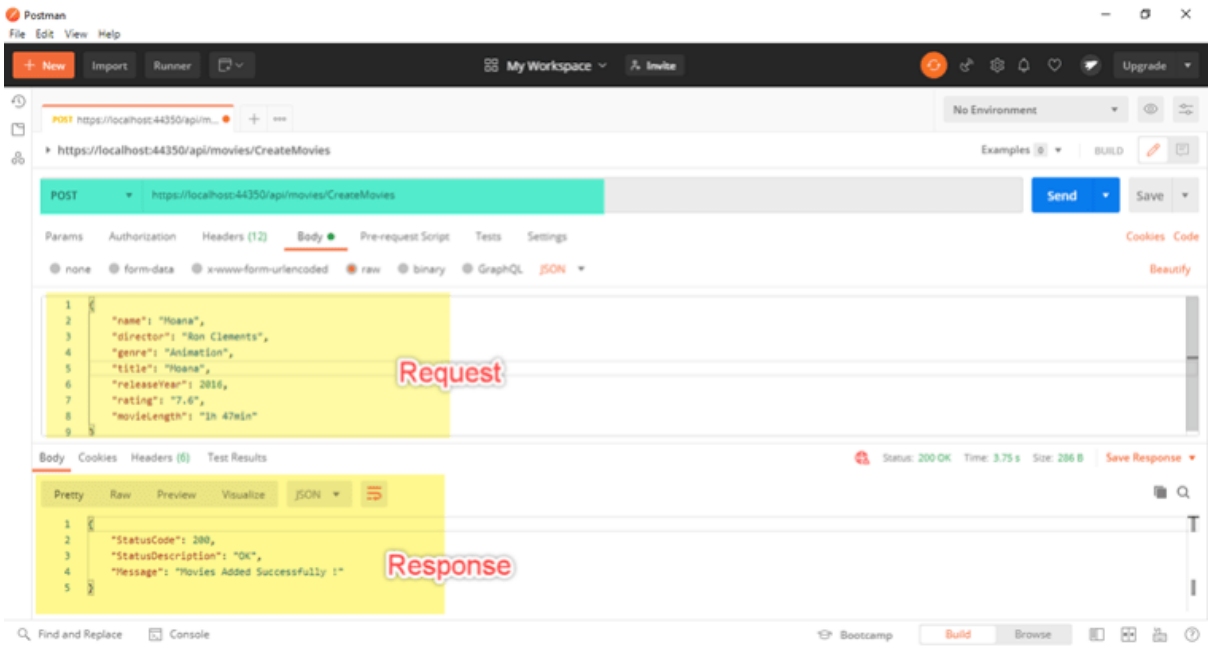
1. **x-frapper-api-version** : – Version of API 1 or 2
2. **Authorization**: – Jwt token
3. **Content Type**: – application/json



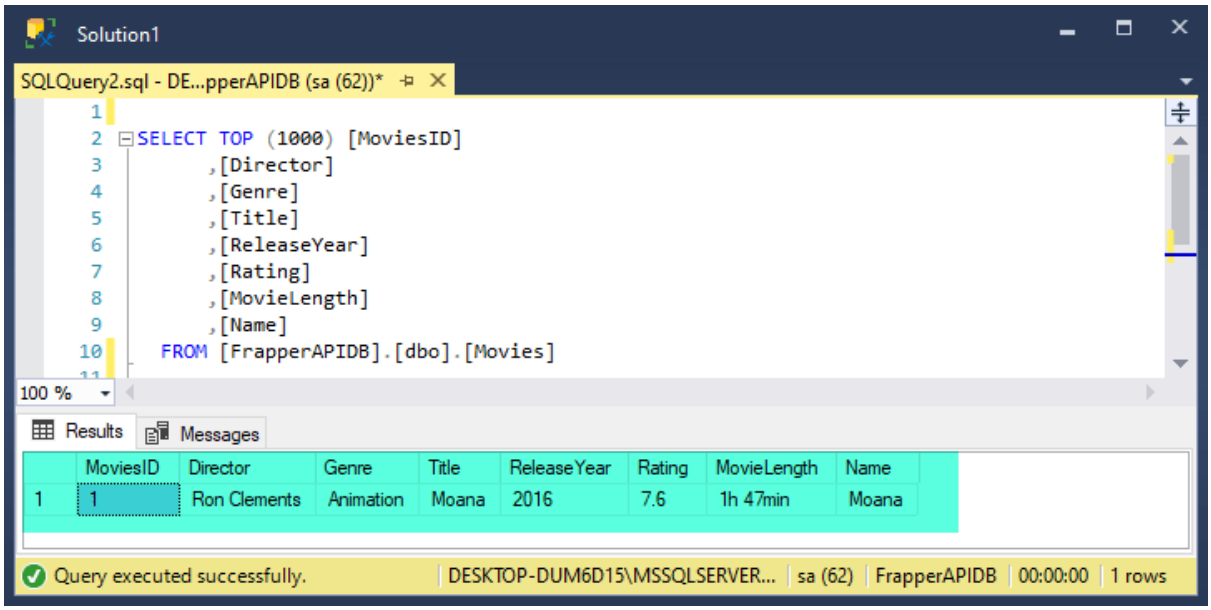
- Setting Headers Request



Response



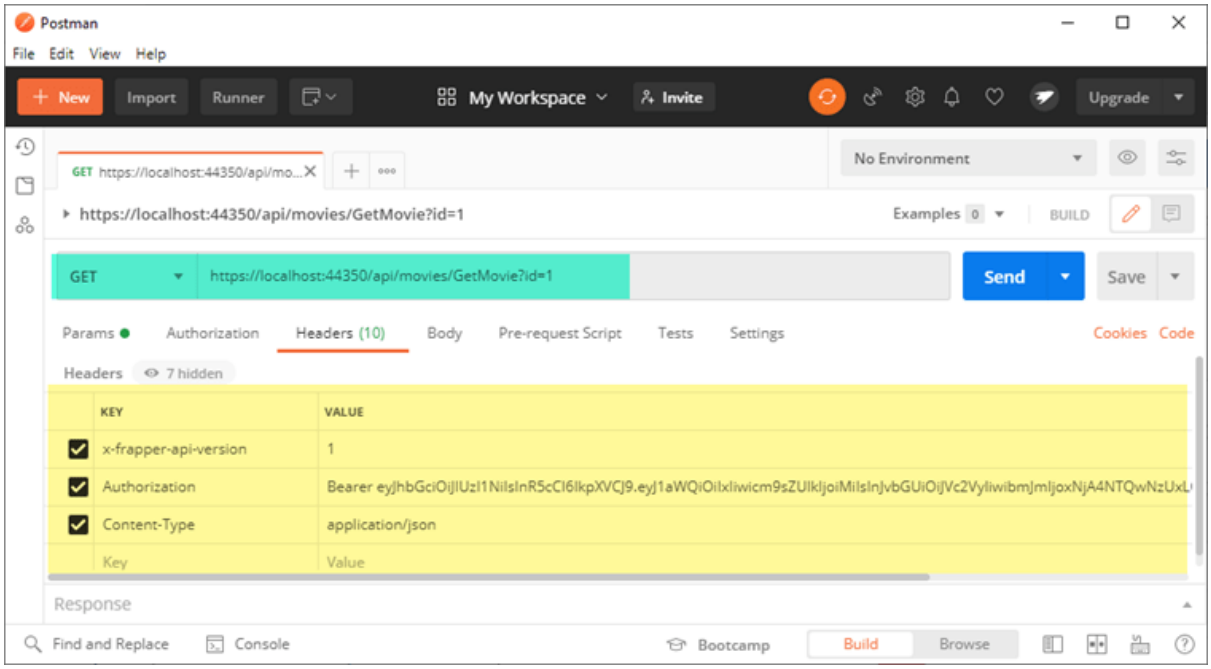
Output



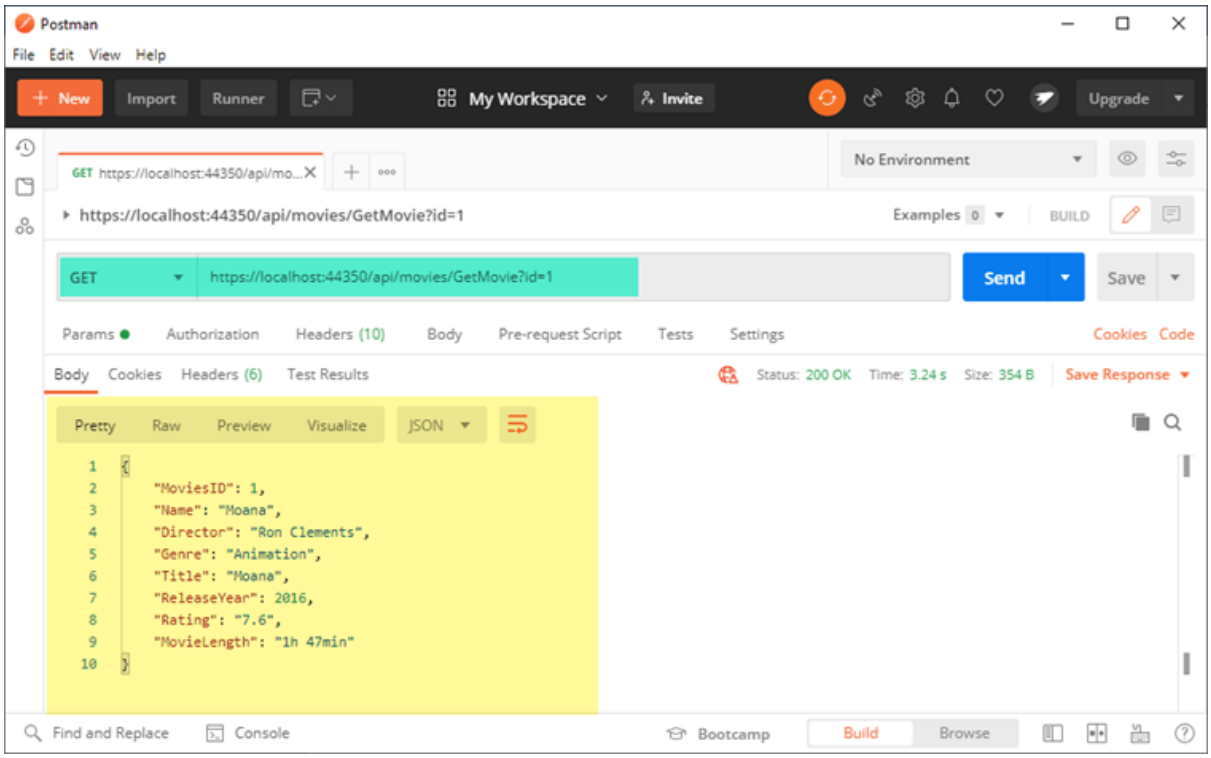
Get Movie Details by Id

In this part, we will get Movie details that we have created by sending Id (**MoviesID**). While getting details also we need to send Headers which we have sent while creating Movie.

Request



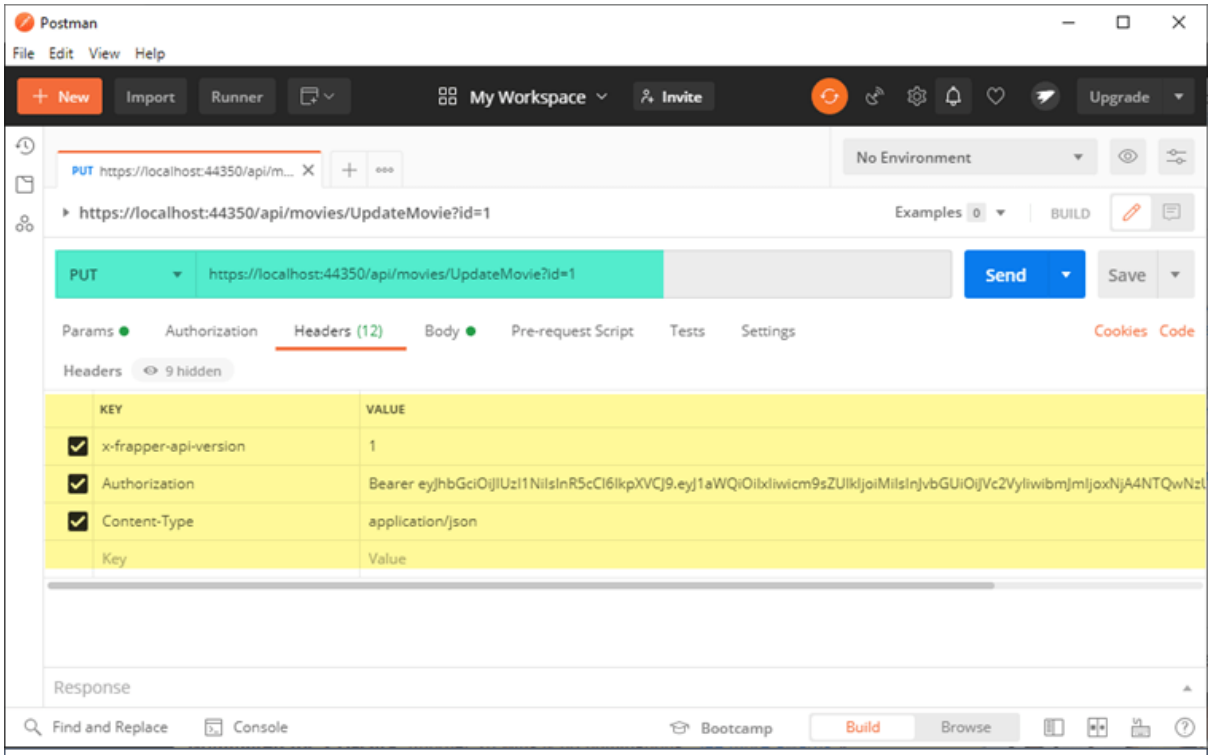
Response

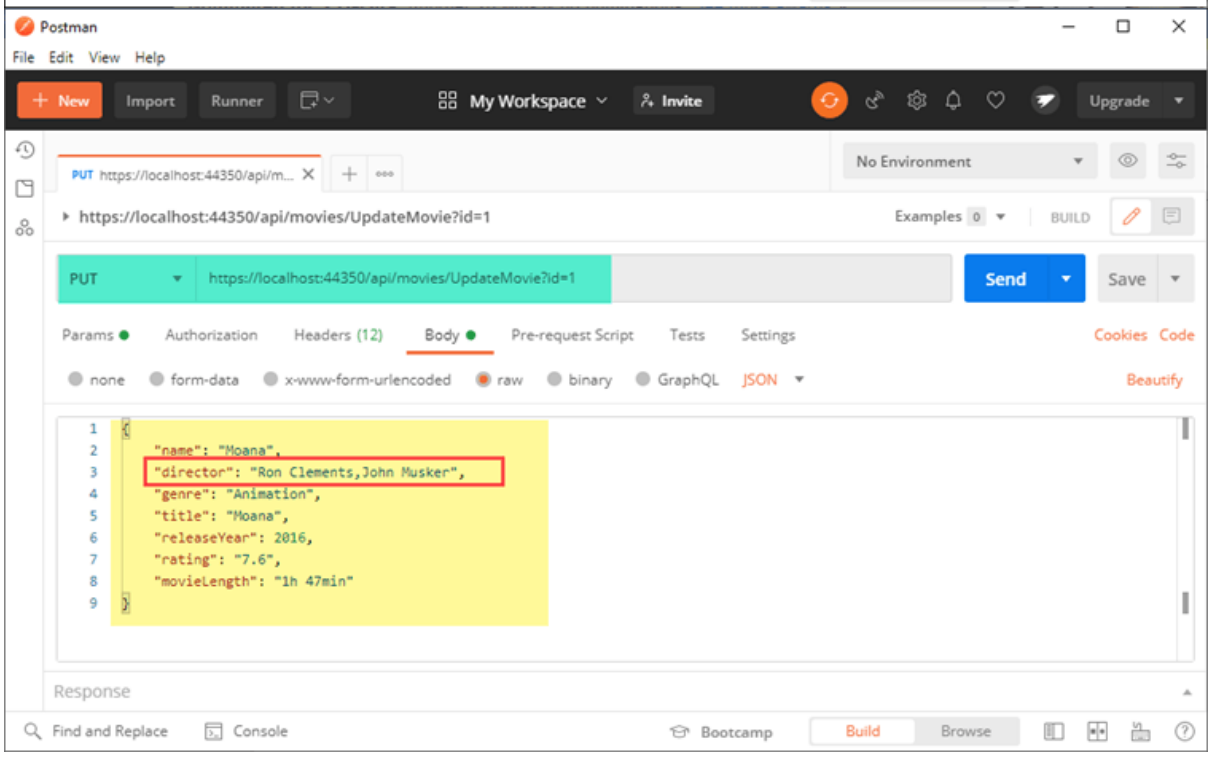


Updating Movie Details

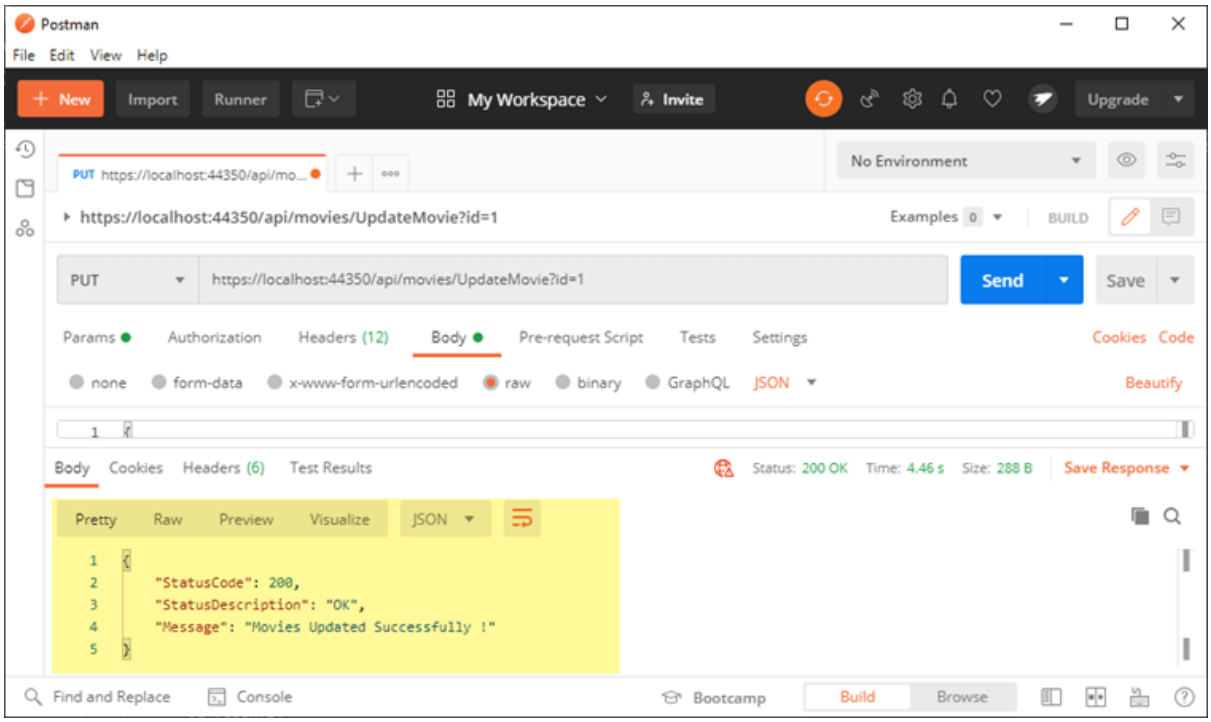
In this part, we are going to Update Movie Details. For doing an update, we are going to use **PUT** HTTP request.

Request

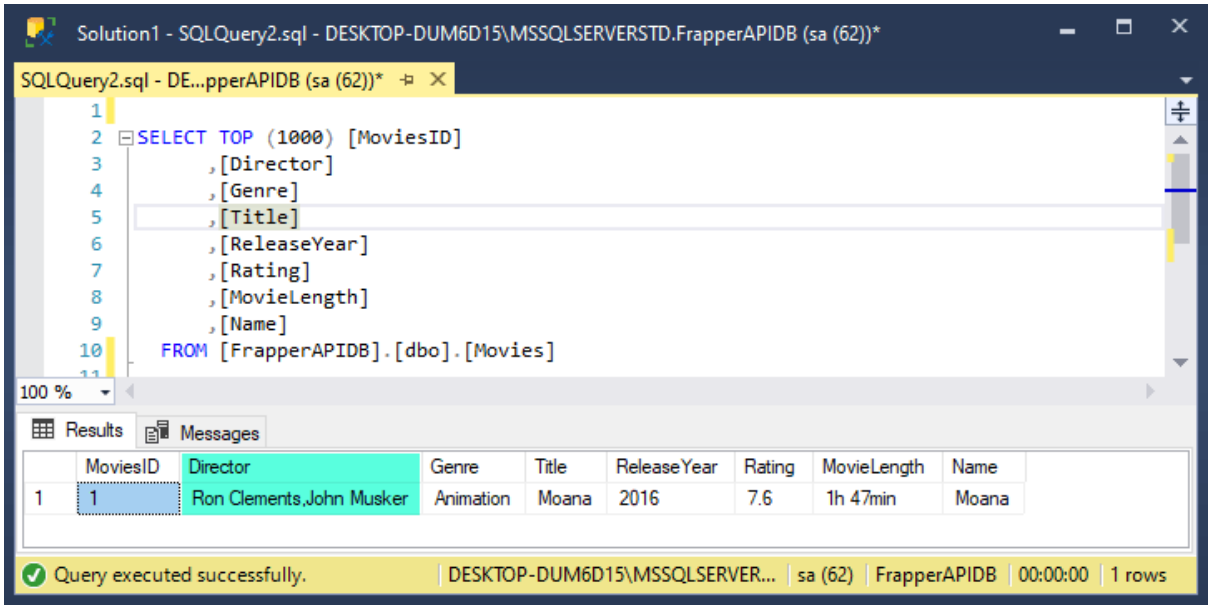




Response



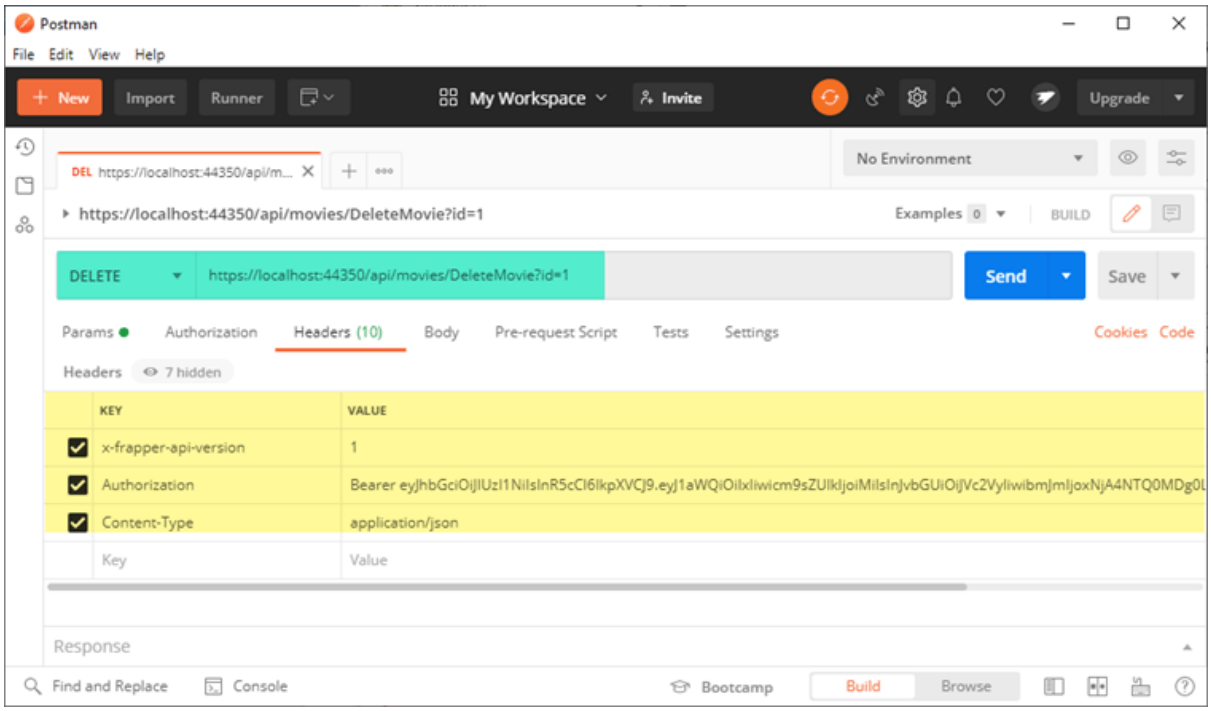
Output



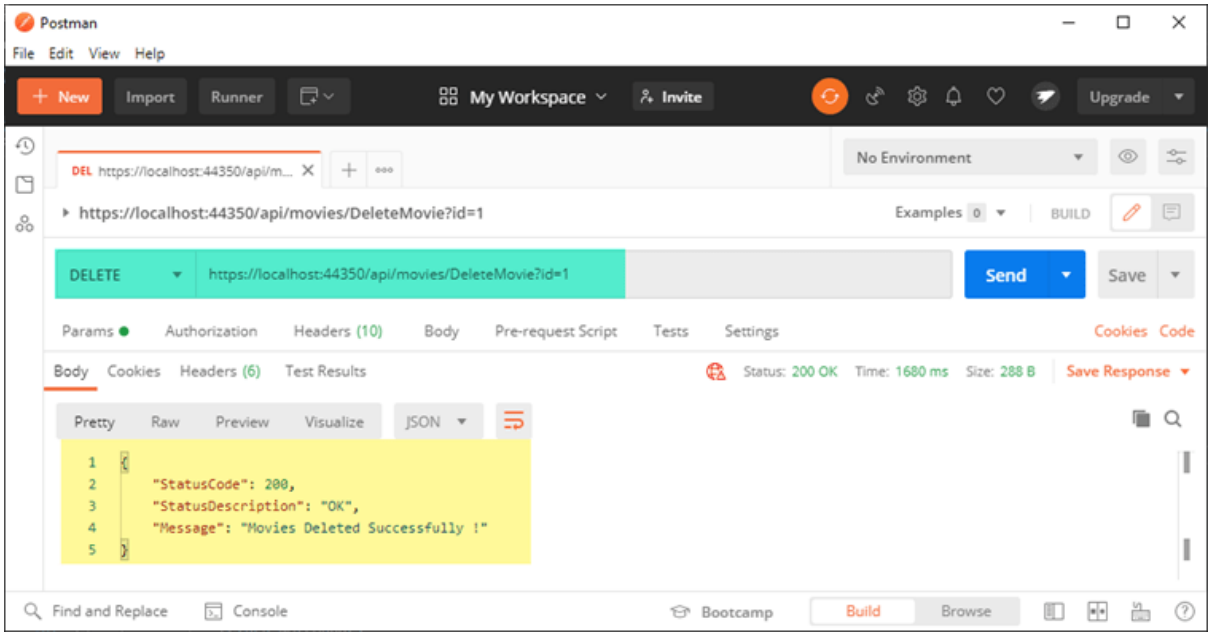
Delete Movie Details

In this part, we will delete Movie Details for doing that we will use **Delete** HTTP request.

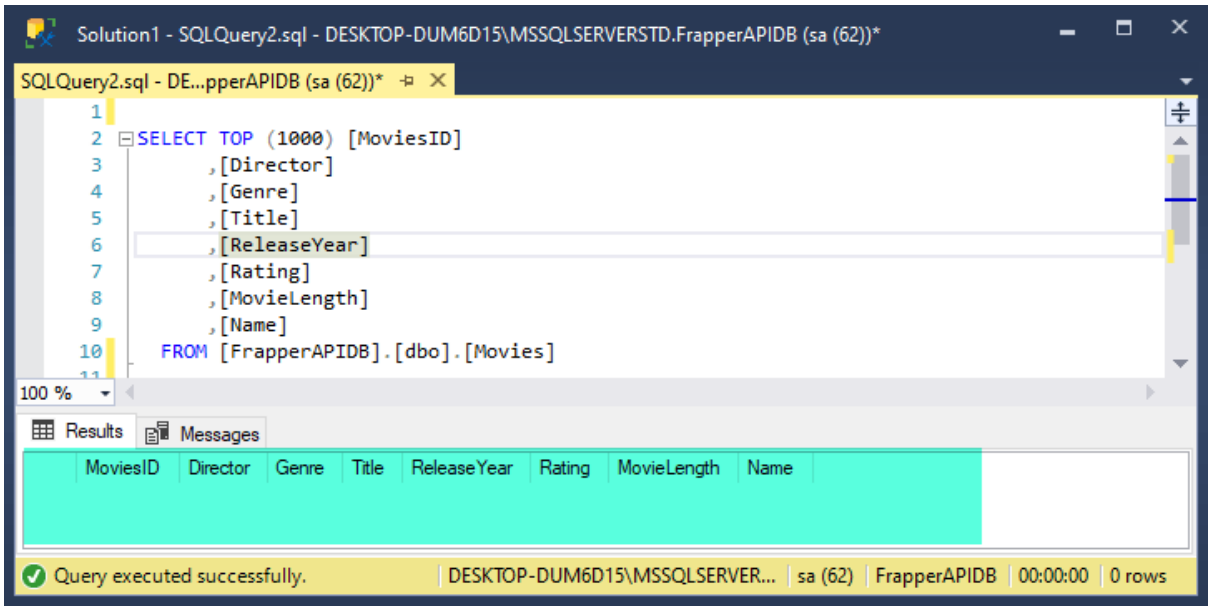
Request



Response



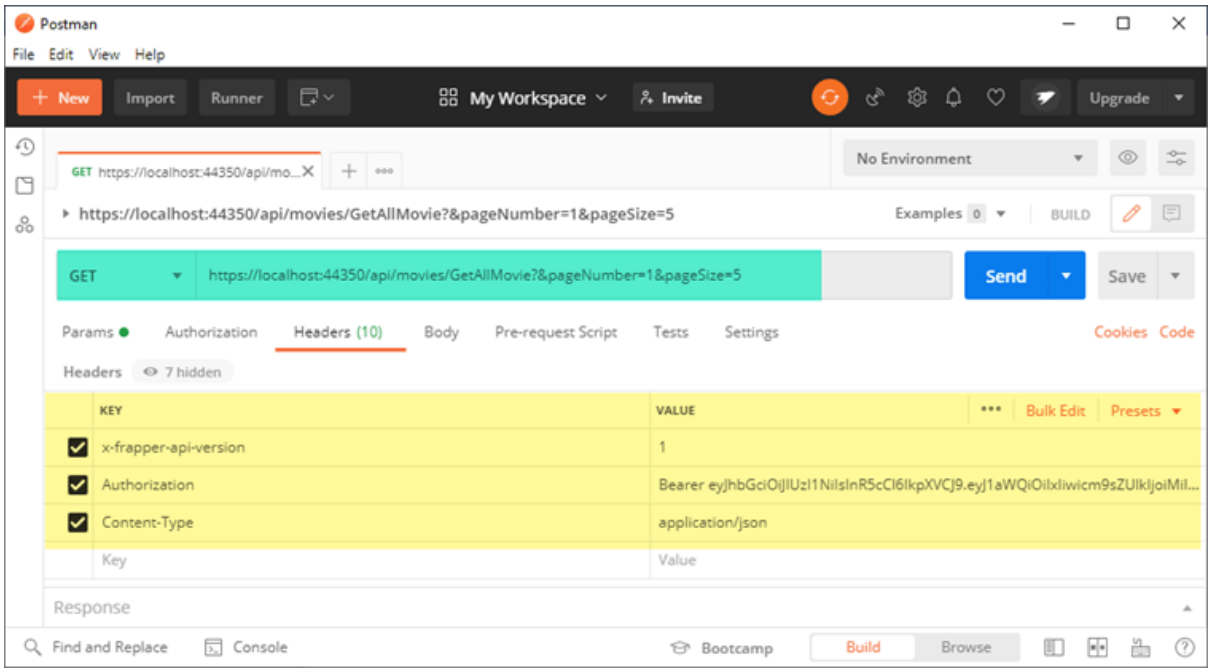
Output



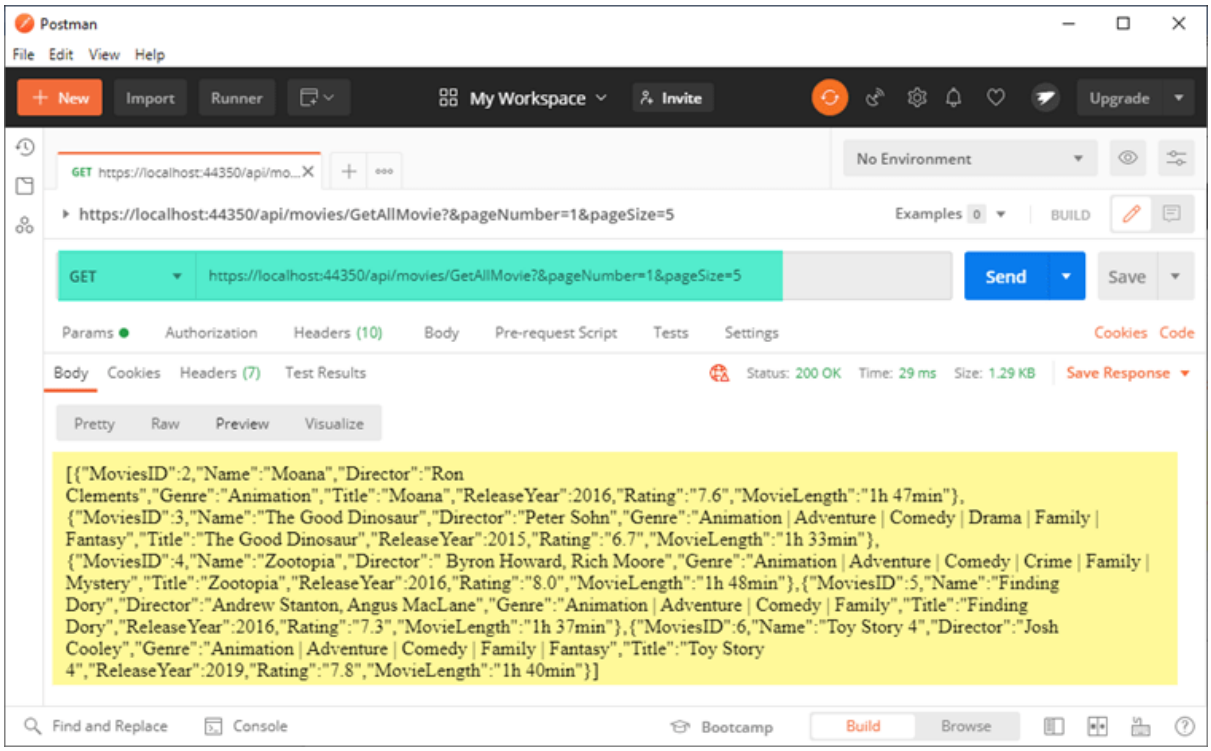
Paging

In this part, we will get all Movies Details for doing that we will use paging where we are going to send page number and pagesize parameters in URL according to it we are going get details.

Request



Response

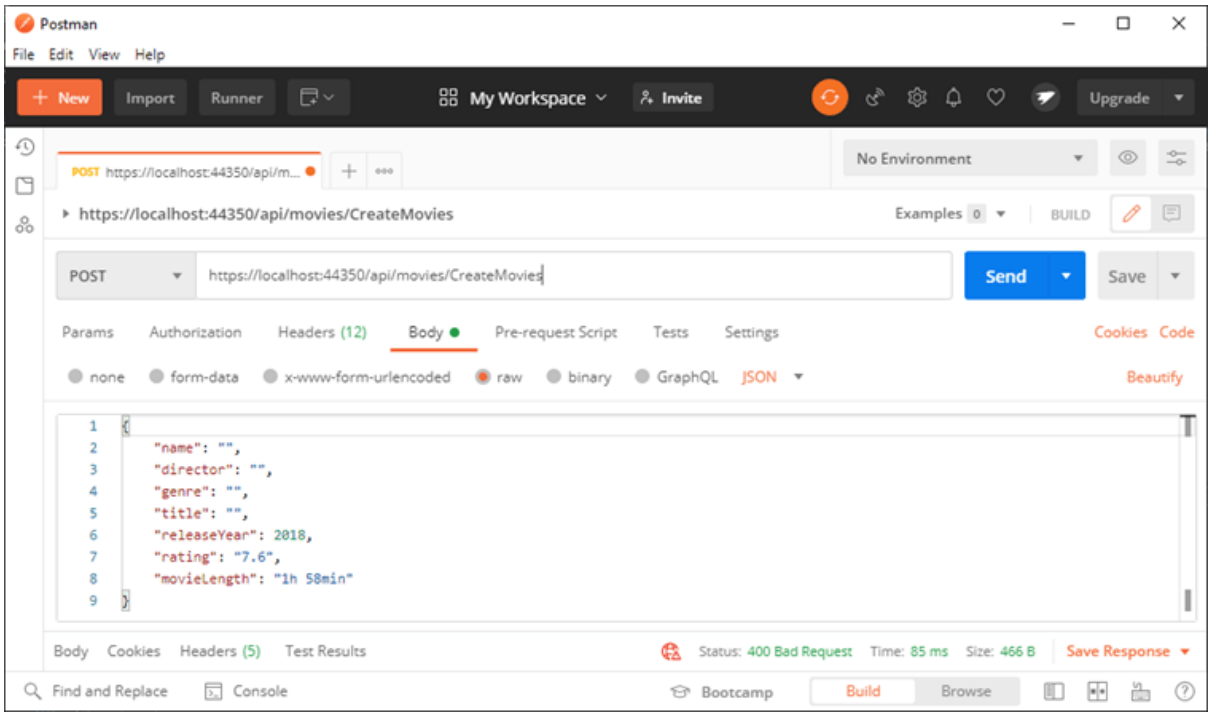


Validation

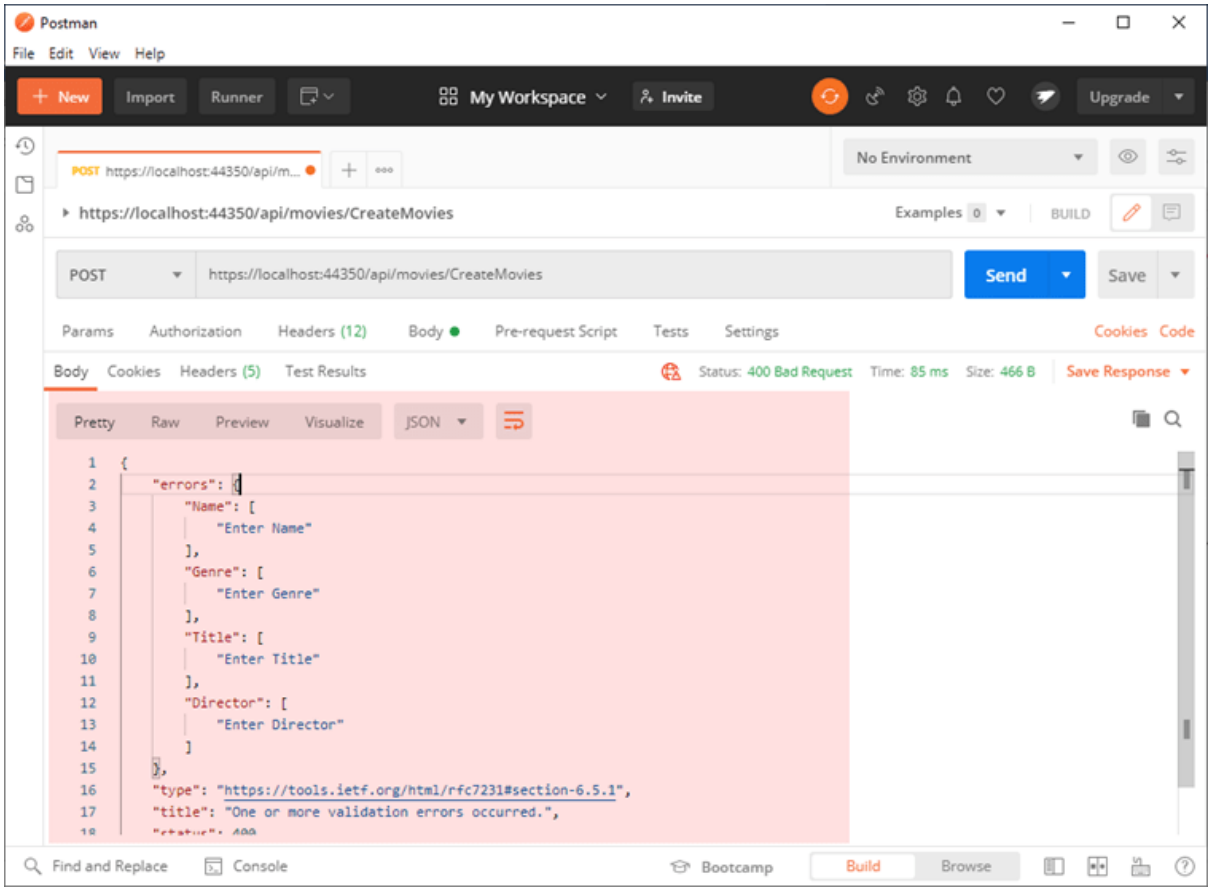
In creating the Movie Model, we have kept all fields mandatory. While we are sending request; we need all fields if it is not there then it will show an error message. For validating we have created ValidateModel Attribute which is an ActionFilter Attribute in this Attribute we are going validating model state if it is invalid then we are returning errors.

```
using System.ComponentModel.DataAnnotations;
namespace Frapper.ViewModel.Movies.Request
{
    public class MoviesCreateRequest
    {
        [Required(ErrorMessage = "Enter Name")]
        [Display(Name = "Name")]
        public string Name { get; set; }

        [Required(ErrorMessage = "Enter Director")]
        [Display(Name = "Director")]
        public string Director { get; set; }

        [Required(ErrorMessage = "Enter Genre")]
        [Display(Name = "Genre")]
        public string Genre { get; set; }

        [Required(ErrorMessage = "Enter Title")]
        [Display(Name = "Title")]
        public string Title { get; set; }

        [Required(ErrorMessage = "Enter ReleaseYear")]
        [Display(Name = "ReleaseYear")]
        public int ReleaseYear { get; set; }

        [Required(ErrorMessage = "Enter Rating")]
        [Display(Name = "Rating")]
        public string Rating { get; set; }

        [Required(ErrorMessage = "Enter MovieLength")]
        [Display(Name = "MovieLength")]
        public string MovieLength { get; set; }
    }
}
```

Request



Response

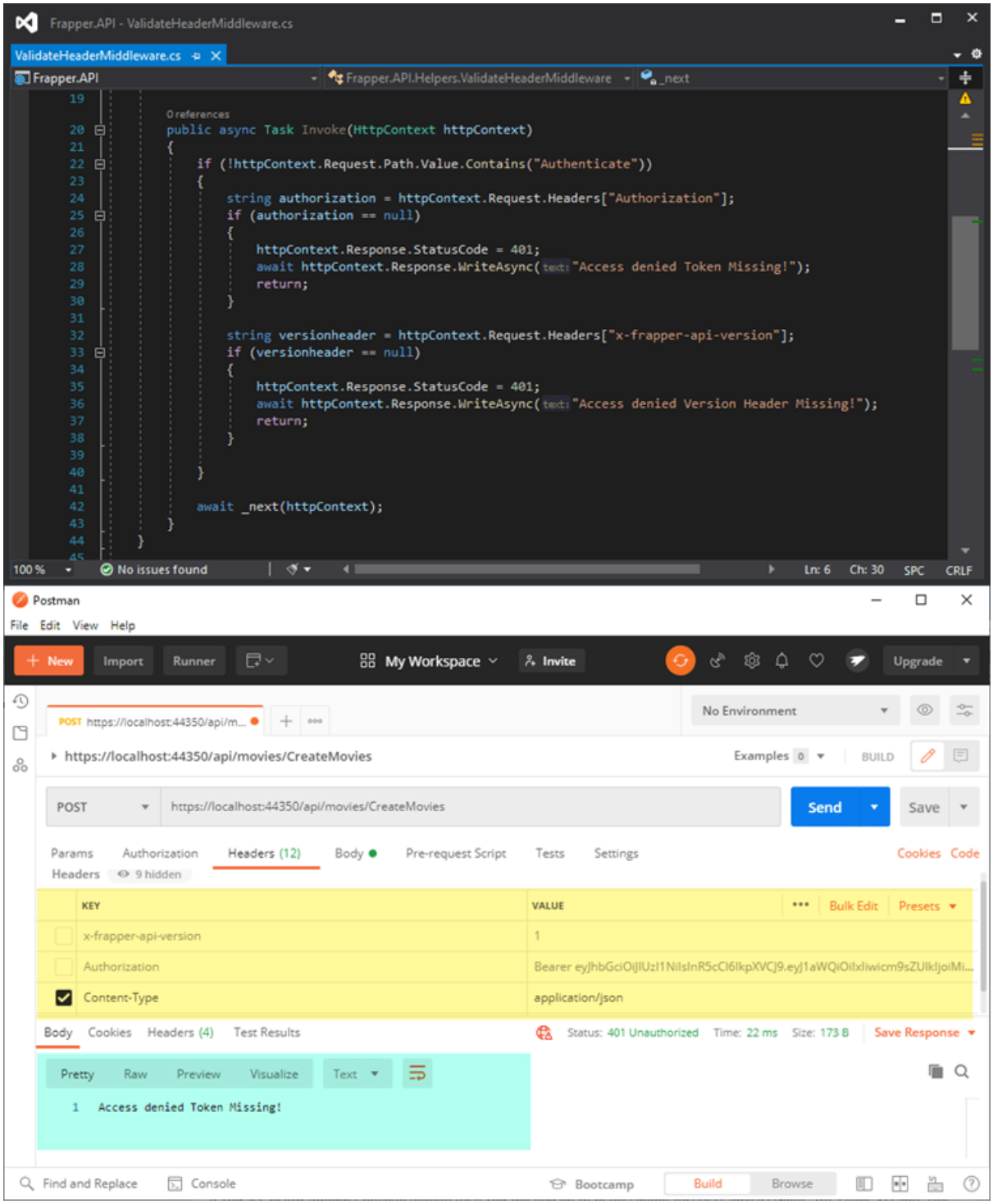


Custom Middleware

In this application, we have written 2 custom middleware

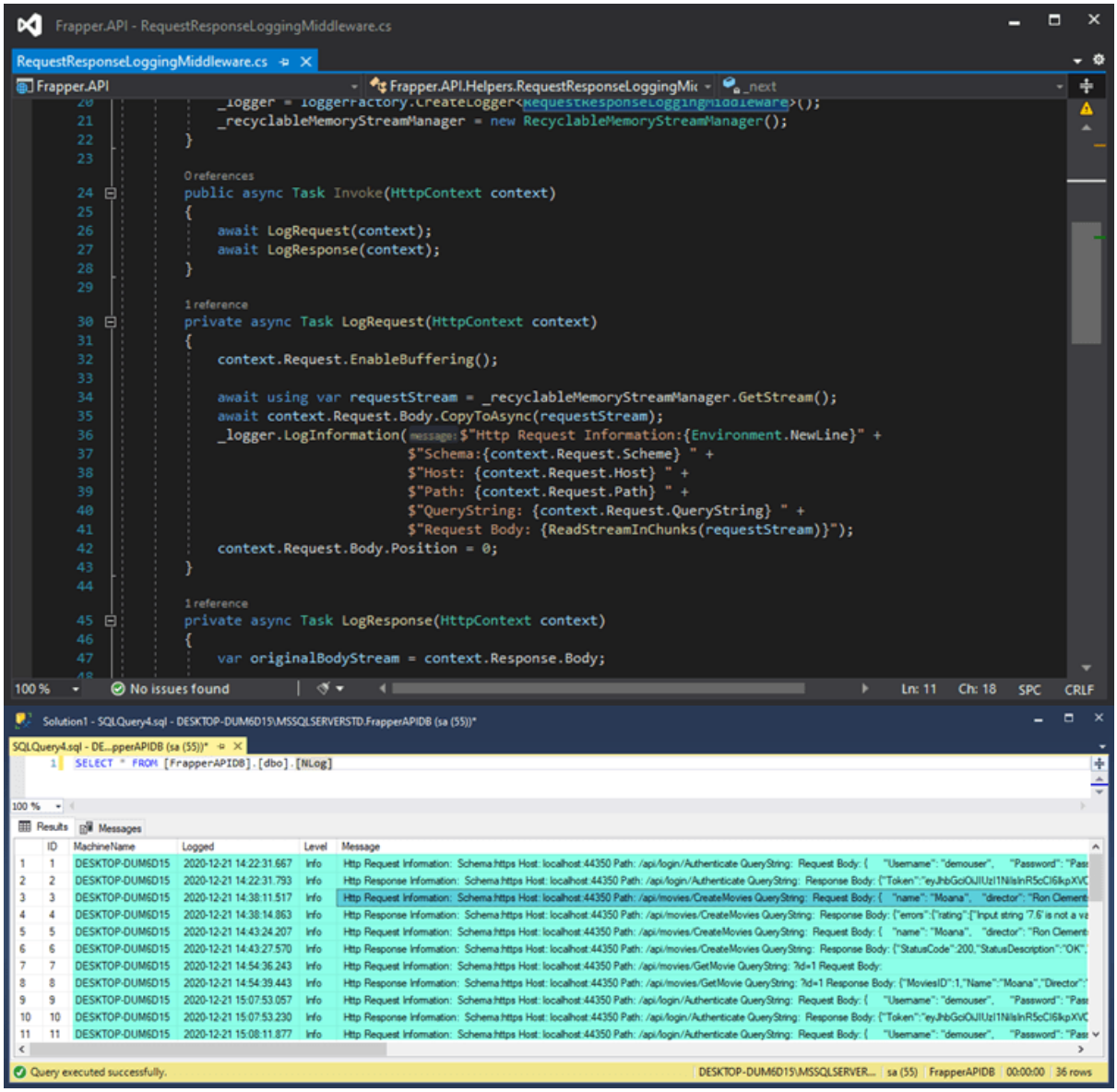
- Validating Headers
- Logging Request and Response
- Validating Headers

Each request other than Authenticate should have Authorization and API Version header if it is not there, it will show an error.



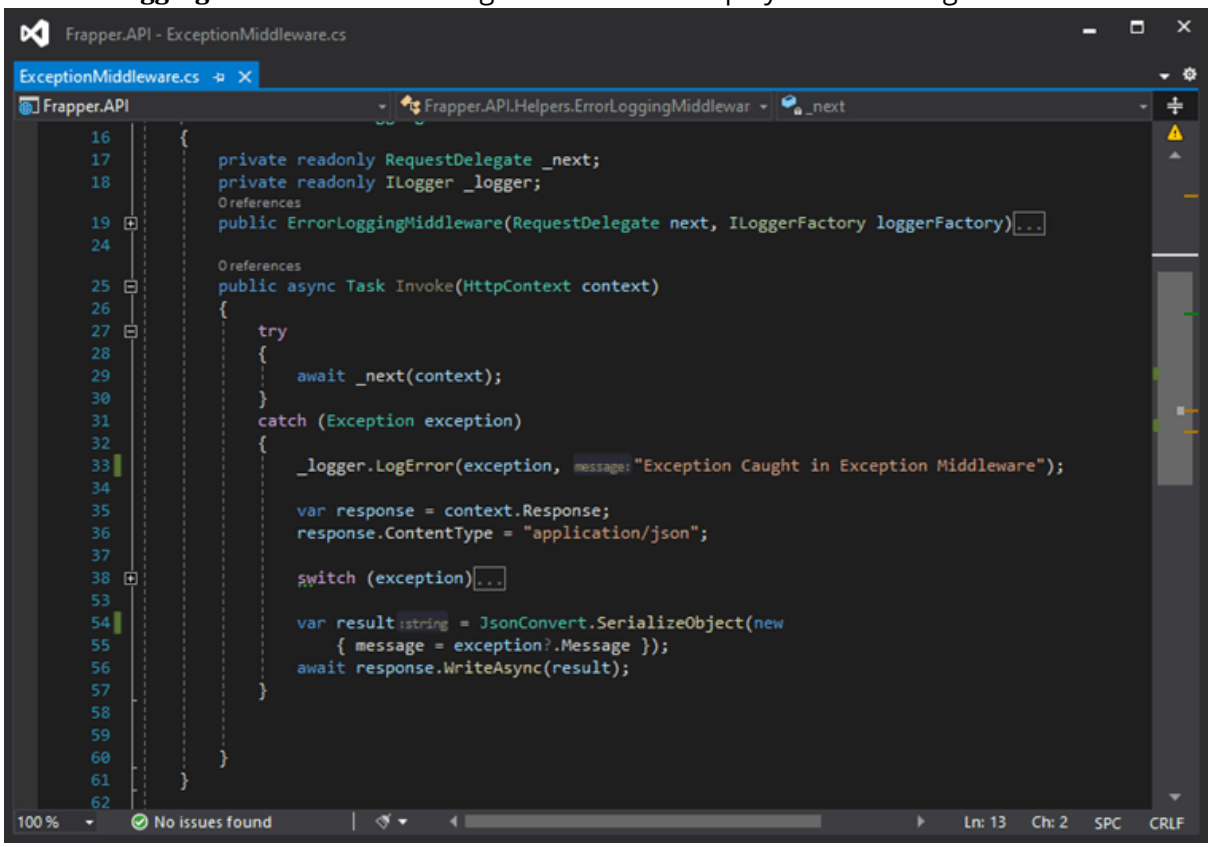
- **Logging Request and Response**

Logging Request and Response is also middleware which keeps logging each request.

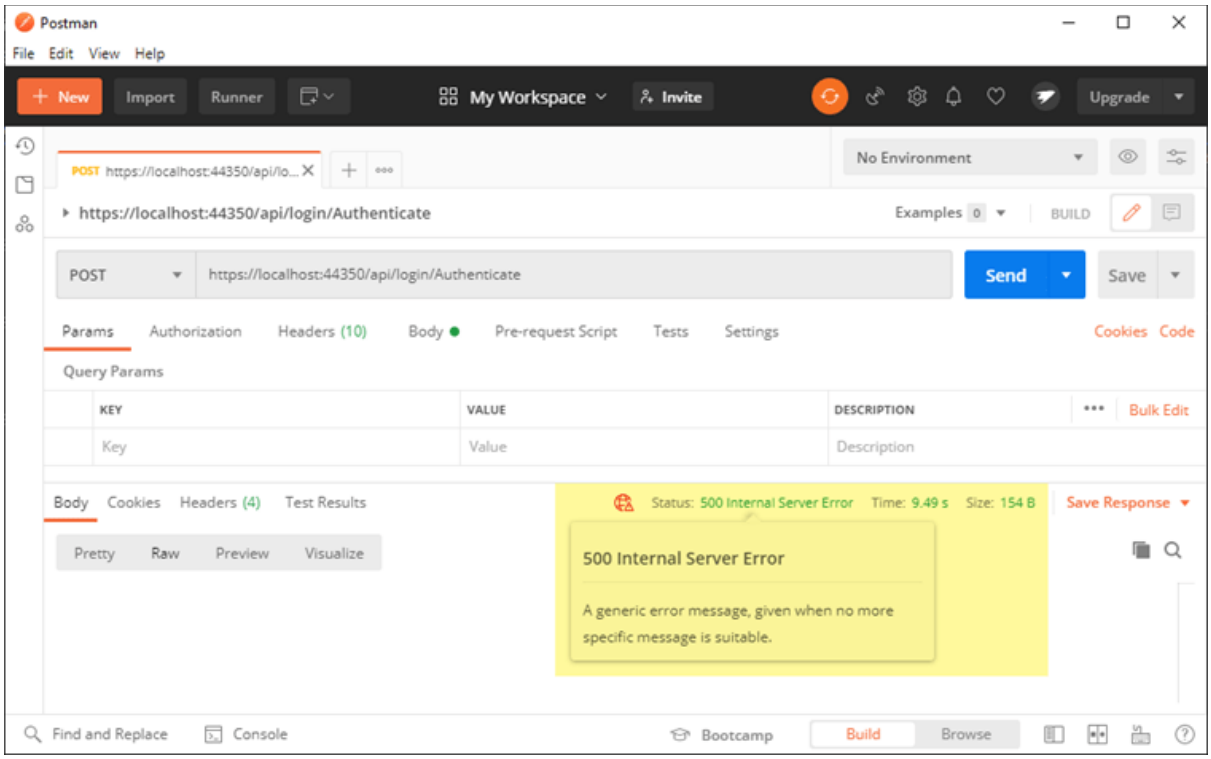


Exception Handling and Error Logging

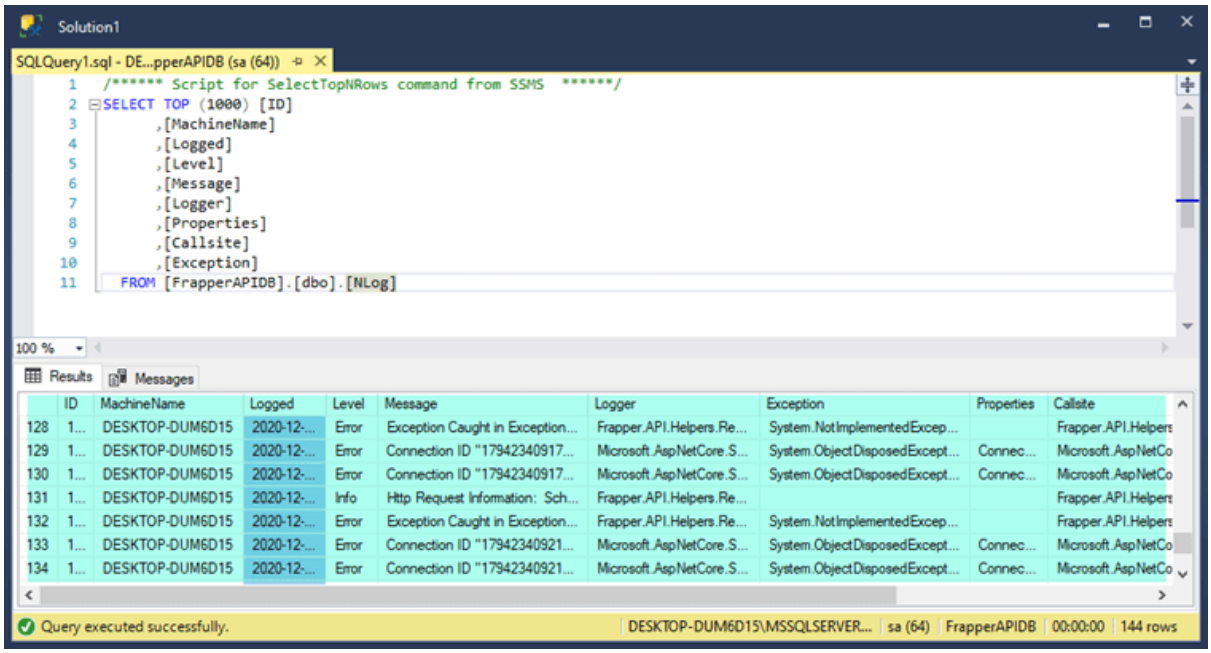
For exception handling and logging error, we have used middleware. Below is an example of **ErrorLoggingMiddleware** which logs an error and display Error Message.



If you see below the POST window, we have sent a request that throws an error, but we do not show error details to the user but show standard status code.

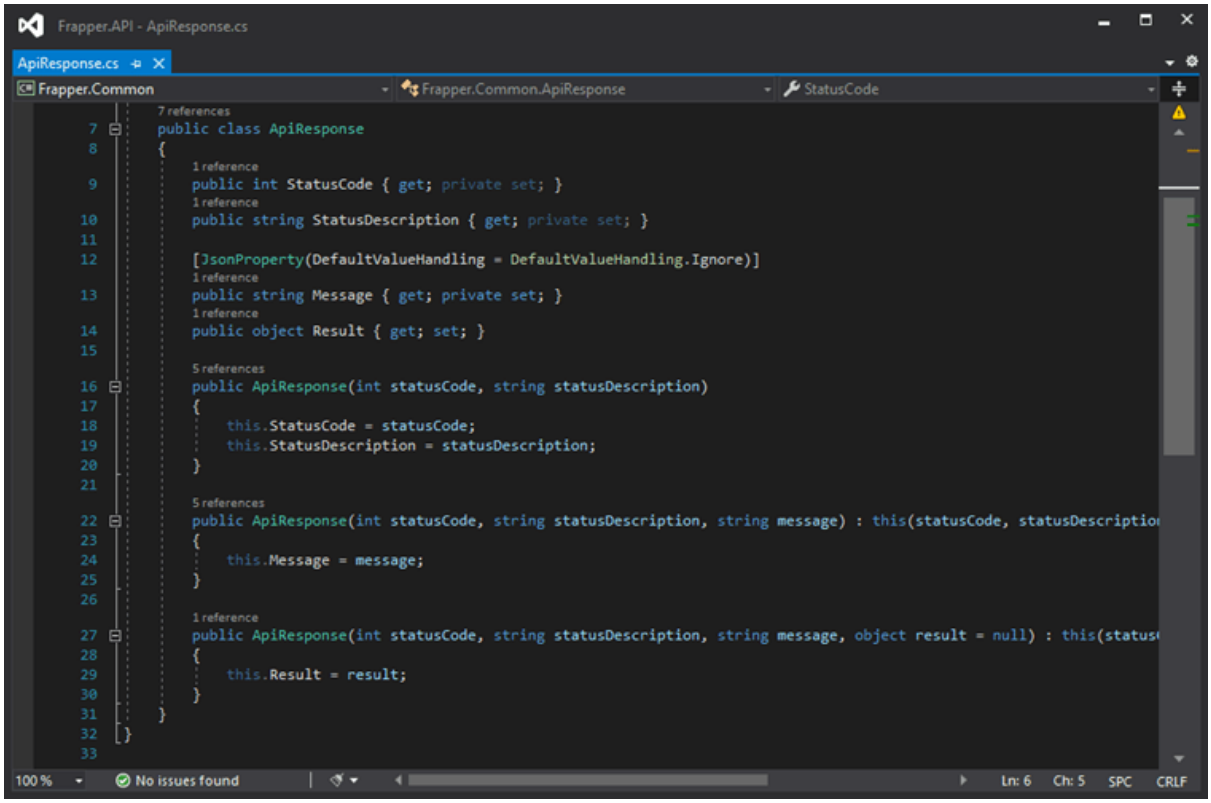


Displaying Logged Errors in Database



Common Response

We have tried to create a common response in an entire application for that we have created base Class ApiResponse. ApiResponse class is inherited by different class such as **OkResponse**, **BadRequestResponse**, **NotFoundResponse**. While sending a response, we use this object to create a common response



```
if (_userMasterQueries.CheckUserExists(createUserRequest.UserName))
{
    return BadRequest(error:new BadRequestResponse(message:"Entered Username Already Exists"));
}

if (_userMasterQueries.CheckEmailExists(createUserRequest.EmailId))
{
    return BadRequest(error:new BadRequestResponse(message:"Entered EmailId Already Exists"));
}

if (_userMasterQueries.CheckMobileNoExists(createUserRequest.MobileNo))
{
    return BadRequest(error:new BadRequestResponse(message:"Entered MobileNo Already Exists"));
}

if (!string.Equals(a:createUserRequest.Password, b:createUserRequest.ConfirmPassword,
    StringComparison.Ordinal))
{
    return BadRequest(error:new BadRequestResponse(message:"Password Does not Match"));
}
```

Response

Pretty Raw Preview Visualize JSON

```
{
    "StatusCode": 200,
    "StatusDescription": "OK",
    "Message": "Success",
    "Result": {
        "Token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
            eyJ1aWQiOiIxIiwicm9sZU1kIjoiMiIsInJvbGUiOiJVc2VyIiwibmJmIjoxNjA4NjQzMDEyLCJleHAiOjE2MDg2NDQ4MTIsImlhdCI6MTYwODY0MzAxMiwi
            aXNzIjoiaHR0cHM6Ly9sb2NhbGhvc3Q6NDQzNTAvIiwiYXVkIjoiaHR0cHM6Ly9sb2NhbGhvc3Q6NDQzNTAvIn0.
            VkFnSvOk0gKWj1Jb9OhSgL_Hav3D3BruPO3PYqvPtas"
    }
}
```

Pretty Raw Preview Visualize JSON

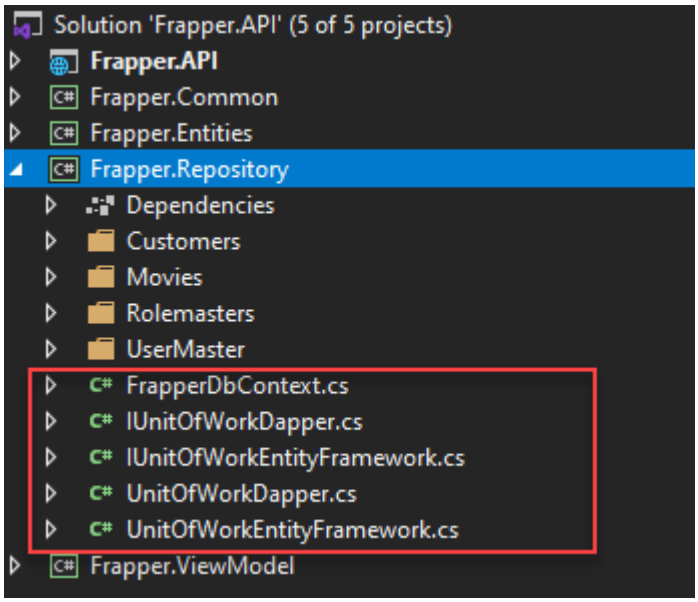
```
{
    "StatusCode": 400,
    "StatusDescription": "BadRequest",
    "Message": "Username or password is incorrect"
}
```

Pretty Raw Preview Visualize JSON

```
{
    "StatusCode": 404,
    "StatusDescription": "NotFound",
    "Message": "Movie Not Found"
}
```

Using Unit of Work with Entity Framework Core and Dapper ORM

In this application, we have used the Unit of Work for Insert, Updating and Deleting record using Entity framework and similar another version of Unit of Work. We have used Dapper ORM for inserting data.




```
[Authorize(Roles = "User")]
[Route(template: "api/movies")]
[ApiVersion("1.0")]
[ApiController]
1 reference
public class MoviesController : ControllerBase
{
    private readonly IUnitOfWorkEntityFramework _unitOfWorkEntityFramework;
    private readonly IMoviesQueries _iMoviesQueries;

    0 references
    public MoviesController(IUnitOfWorkEntityFramework unitOfWorkEntityFramework, IMoviesQueries moviesQueries)
    {
        _unitOfWorkEntityFramework = unitOfWorkEntityFramework;
        _iMoviesQueries = moviesQueries;
    }
```

Reference : https://github.com/saineshwar/Frapper.API